CPLREADER - A CFD DATA READER AND ACOUSTIC SOURCE TERM GENERATOR FOR CFS++

Max Escobar and Jens Grabinger

mailto:jens.grabinger@lse.eei.uni-erlangen.de

Chair of Sensor Technology
Friedrich-Alexander University of Erlangen-Nuremberg, Germany

Simon Triebenbacher and Stefan Zörner

mailto:simon.triebenbacher@uni-klu.ac.at

Chair of Applied Mechatronics
Alps-Adriatic University of Klagenfurt, Austria

Contents

1 Name

cplreader - A CFD Data Reader and Acoustic Source Term Generator for *CFS++*.

The name was invented by Max Escobar , the original author of this program, during his PhD thesis [6].

2 Usage

3 Description

In computational aero-acoustics the so-called hybrid methods are quite common. These methods try to extract source terms for acoustics from CFD data. There exits a wide variety of such methods. The volume based approach used by *CFS++*[1] [11] is the computation of Lighthill source terms in a FEM framework directly on the the CFD mesh. Since the CFD mesh is usually too fine to perform an acoustic computation on it, the source terms have to be conservatively interpolated to a dedicated acoustic grid. The first step is the job of *cplreader*. The interpolation is performed within *CFS++*.

The source term generator *cplreader* is used to read CFD meshes from codes like *ANSYS CFX*[2], *OpenFOAM*[3] or *FASTEST-3D*[4] with the associated velocity, pressure and density fields, to compute the source terms and writing them to *HDF5*[5] files.

Aside from its primary designated use *cplreader* can also be used to convert meshes from the *ANSYS* mesh generator to the *HDF5* file format for use in *CFS++* or *NACS*.

[1] *CFS++* is a FEM solver developed for research purposes at LSE Erlangen and the Chair of Applied Mechatronics at Klagenfurt. There exists also a commercial version called *NACS* which is distributed by Simetris GmbH `http://www.simetris.de`

[2] ANSYS CFX, `http://www.ansys.com/Products/cfx.asp`

[3] OpenFOAM, `http://www.opencfd.co.uk/openfoam`

[4] Flow solver FASTEST-3D developed at LSTM Erlangen

[5] Hierarchical Data Format 5, `http://hdf.ncsa.uiuc.edu/HDF5`

4 Options

Table 1: Command line options of *cplreader*.

Parameter	Default Value	Short Description
--docu		Create directory with detailed documentation (can just be PDF) This option will create the directory `cplreader_docs` in the current working directory and put the file `cplreader_docu.pdf` there.
--name		Name of dataset as well as of the directory containing the dataset This is the name of the dataset to be worked on. For most readers the data files must be placed in a directory of this name, too.
--type	CFX	Type of dataset (can be ANSYS | CFX | CFX_EXPORT | FASTEST | OPENFOAM) Specify the type of dataset to be read: ANSYS see Sec. 8.1, CFX see Sec. 7.2, CFX_EXPORT see Sec. 7.3, FASTEST see Sec. 7.1, OPENFOAM see Sec. 7.4.
--xmlfile		XML file containing options for cplreader. The XML file specified here contains informations for the file readers/writers.

--basedir	.	Base directory where subdirectories are searched.
		The files to be read are searched for in the directory specified by basedir.
		For CFX this means for example:
		
		`basedir/name/`
		`|-- name`
		`|  |-- 1.trn`
		`|  +-- 2.trn`
		`|-- name.def`
		`+-- name.res`
		
		By default the files will be searched for in the current working directory.

--extfiles	true	Use external time step files (just like CFX .trn files). If this parameter is true *cplreader* will create on master .h5 file containing the mesh and just references to the time step files. For each time step an extra *external* .h5 file will be created. If this parameter is false everything goes into one big .h5 file. It may be tricky to copy such big files to USB hard disks, which are usually formatted with a FAT32 file system.
--dim	3	Dimension of fluid data. (can be 2 | 3) This parameter defines the geometrical dimension of the fluid data set. Most readers do not require this parameter. It is however present for the FASTEST reader.
--numsteps	0	Number N of timesteps to read. Default: read all available steps. The number of timesteps to read. By default (0) all available timesteps will be read.
--activeparts	all	Values will only be output on partitions specified by activeparts (all | numbers seperated by ␣ or ; or |). When dealing with very large data sets with many partitions/sub-domains, it is often convenient to read the fields and compute the acoustic source for just a few of them.
--outputfields	acouRhsLoad	Which physical fields should be output ([acouRhsLoad | fluidMechDensity | fluidMechPressure | fluidMechVelocity | fluidMechTKE]). Values may be separated by ␣ or ; or | Sometimes it may be necessary to do some post-processing on the fluid fields to get some understanding of the problem at hand. For this reason it is possible to write the most important fields (velocity, pressure, source terms, turbulence kinetic energy, etc.) to the .h5 files by using the outputfields parameter.
--firststep	1	Index of first time step (only for CFX_EXPORT, FASTEST) To start reading at a different time step than the first one this parameter can by used. It is just implemented for CFX_EXPORT and FASTEST at the moment.
--stepincr	1	Step increment for reading the files Increment time step counter by this value und just read each stepincr-th time step.
--timestep	-1.0	Time step length T in seconds This parameter specifies the length of each timestep in seconds.
--calcsrc	false	Calculate the acoustic sources from velocity If this parameter is set a velocity field has to present in each time step from which the Lighthill source term will be computed.
--geomscale	1.0 1.0 1.0	Scale factors for geometry in the format factorX factorY factorZ

Continued on next page

Table 1 – continued from previous page

Parameter	Default Value	Short Description
		This parameter specifies the factors to be multiplied with the corresponding components of the nodal coordinates. This can be useful, if the CFD computation used a different scaling (cf. OpenFOAM simulations of the Freiberg guys)
--velscale	1.0 1.0 1.0	Scale factors for velocity field in the format factorX factorY factorZ This parameter specifies the factors to be multiplied with the corresponding components of the velocity field. This can be useful, if the CFD computation used a different scaling (cf. OpenFOAM simulations of the Freiberg guys)
--density	1.0	Density of the fluid. Default density is 1.0 for testing. The mean density of the fluid may be specified by this parameter. Some practically relevant values include: air $1.2041 kg/m^3$ (at $20°C$ and $101.325 kPa$), water $998.2 kg/m^3$ (at $20°C$)
--segfault	true	Cause program to segfault if an exception is encountered. This parameter is especially important for debugging purposes and should not be necessary in standard runs.
--deltemp	true	Delete temporary files. Some file readers (ANSYS) create or use temporary files. This parameter determines, whether those files will be deleted or not.
--outprec	double	Write data as single or as double precision (can be single | double) With outprec the user can specify the precision with which the floating point numbers will be written to the .h5 files. specifying *single* here may save some space but the user has to be sure what he is doing!
--justinit	false	Just perform initialization step. Specifying justinit will let *cplreader* just perform the init phase. This comes in handy when developing a new reader.
--justmesh	false	Just convert the mesh. Specifying justmesh will make *cplreader* just read the mesh without any other data.
--degen	false	Allow degenerated element shapes. This option just applies to the ANSYS reader. If true all elements other than lines will degenerated version of quadrilaterals and hexahedrons.
--strict	false	Be strict. For the ANSYS reader this option means that *cplreader* will complain about holes in the node list and connectivity, i.e. if the user forgot to nummrg the entities in ANSYS. For the CFX reader this option means that if corrupt transient files have been found, the reader will complain and stop working.
--verbose	false	Be verbose If this parameter is set to true lots of helpful information will be output which is not necessery to have in most cases and would normally just clutter the shell.
--deffile		Definition file name. Only for CFX! Non-standard path to .def file for CFX.
--trntol	0.05	Ratio by which .trn files may differ. Ratio by which .trn files may differ before they are regarded corrupt. E.g.: Let's assume the mean size of .trn files is 1MB. By specifying –trntol 0.05 the files may vary in size from 0.95MB to 1.05MB before an error condition is assumed.
--lhsrc		Column of Lighthill source term in FASTEST result files (e.g. col1). If FASTEST already calculated the Lighthill source term, the column in the ASCII file can be specified using this parameter.
--vx		Column of x-velocity in FASTEST result files (e.g. col2). Specify column of x-velocity in FASTEST ASCII files.
--vy		Column of y-velocity in FASTEST result files (e.g. col3). Specify column of y-velocity in FASTEST ASCII files.
--vz		Column of z-velocity in FASTEST result files (e.g. col4). Specify column of z-velocity in FASTEST ASCII files.
--pres		Column of pressure in FASTEST result files (e.g. col5). Specify column of pressure in FASTEST ASCII files.

5 Theory

The contents of this section have been taken or are based upon in large parts from [12].

5.1 Introduction

A large amount of the total noise in our daily life is generated by turbulent flows (e.g. airplanes, cars, air conditioning systems, etc.). The physics behind the generation process is quite complicated and still not fully understood. The use of numerical simulation tools is one important way to analyze the generation of flow induced sound. Fig. 5.1 displays the general situation for the numerical simulation, we are concerned with, when performing computational aeroacoustics. Therein Ω_1 denotes the computational domain, where we have to solve for the flow field and the generation of sound, and Ω_2 the domain where the flow is zero, and where we are interested in the radiated sound.

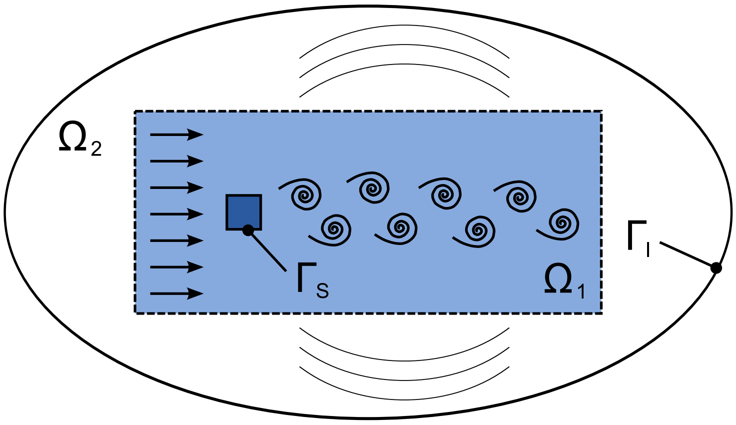


Figure 1: General setup of aeroacoustics

The surface Γ_S defines the interaction between structural mechanics and flow and Γ_I the boundary of the total computational domain. Now, the computation of the flow induced noise exhibits a lot of difficulties, where the most significant ones can be summarized as follows:

1. The scale discrepancy between flow pressure and acoustic pressure reaches values of 10^7.
2. The acoustic domain is in most cases large compared to the fluid flow domain.
3. The spatial discretization for the flow computation is much finer as compared to the discretization of the acoustic domain.
4. In many applications, the interface between flow and solid/elastic bodies leads to vibrations, which not only effect the flow, but also emits acoustic sound (vibrational acoustics).

Due to 1-3, a direct numerical simulation (DNS) of Navier Stokes equations, which are capable to describe both the flow as well as acoustic field, is restricted to very few practical cases (see e.g., [16]). Therefore, most approaches, published in the last 20 years, uses hybrid methods, e.g. [17], [3], [7]. Since we concentrate on a computational scheme, which allows the computation of the flow generated sound simultaneously with any flow induced vibrational sound (due to solid-fluid interactions), we base our scheme on Lighthill's analogy [13, 14]. This formulation will provide a direct coupling between flow and acoustic sound computation in the time domain and in addition will offer for both physical fields different discretization. At this point, we want to notice, that a similar approach can be found in [15], which however has just been investigated in the time-harmonic case.

The outline of this theory section is as follows: First of all, we give a review of Lighthill's analogy (Sec. 5.2) and its generalization. In Sec. 5.3 we will then concentrate on the weak formulation of Lighthill's analogy and its finite element (FE) formulation. Section 5.4 provides a description of our environment for computational aeroacoustics.

5.2 Lighthill's Analogy and its Extension

Let us start our derivation of Lighthill's analogy by introducing the two main equations of fluid dynamics: mass conservation and momentum conservation. The differential and coordinate-free formulation of the mass conservation (continuity) equation reads as follows

$$\frac{\partial \rho}{\partial t} + \frac{\partial \rho v_i}{\partial x_i} = 0\,. \tag{1}$$

In (Eq. (1)) ρ denotes the density of the fluid and v_i the i-th component of the flow velocity vector $\mathbf{v}$. The momentum conservation equation is expressed by

$$\rho \frac{\partial v_i}{\partial t} + \rho v_j \frac{\partial v_i}{\partial x_j} = -\frac{\partial p}{\partial x_i} - \frac{\partial \tau_{ij}}{\partial x_j} = -\frac{\partial}{\partial x_j}\left(p\delta_{ij} + \tau_{ij}\right)\,, \tag{2}$$

where p denotes the total pressure within the flow, $[\tau]$ the viscous stress tensor and δ the Kronecker symbol. For Newtonian fluids, we can express $[\tau]$ by

$$\tau_{ij} = -\mu\left(\frac{\partial v_i}{\partial x_j} + \frac{\partial v_j}{\partial x_i}\right) + \frac{2}{3}\mu\delta_{ij}\frac{\partial v_k}{\partial x_k} \tag{3}$$

with μ the dynamic viscosity.

In a first step, we multiply (Eq. (1)) by v_i (before we substitute the index i by j) and add the so obtained equation to (Eq. (2)). In addition, we substitute p in (Eq. (2)) by $p - p_0$ with p_0 the mean pressure. This operation is allowed, since the relation $\partial(p-p_0)/\partial x_j = \partial p/\partial x_j$ holds. Therewith, we obtain

$$\underbrace{\rho\frac{\partial v_i}{\partial t} + v_i\frac{\partial \rho}{\partial t}}_{\dfrac{\partial \rho v_i}{\partial t}} + \underbrace{\rho v_j\frac{\partial v_i}{\partial x_j} + v_i\frac{\partial \rho v_j}{\partial x_j}}_{\dfrac{\partial}{\partial x_j}(\rho v_i v_j)} = -\frac{\partial}{\partial x_j}((p-p_0)\delta_{ij} - \tau_{ij}) \tag{4}$$

By introducing the momentum flux tensor $[\tau^{\mathrm{I}}]$ by

$$\tau^{\mathrm{I}}_{ij} = \rho v_i v_j + (p-p_0)\delta_{ij} - \tau_{ij} \tag{5}$$

we may write (Eq. (4)) as

$$\frac{\partial \rho v_i}{\partial t} = -\frac{\partial \tau^{\mathrm{I}}_{ij}}{\partial x_j}\,. \tag{6}$$

For linear acoustics, the momentum flux tensor $[\tau^{\mathrm{I}}]$ reduces to

$$\tau^0_{ij} = (p-p_0)\delta_{ij} \tag{7}$$

and (Eq. (6)) reads as

$$\frac{\partial \rho v_i}{\partial t} + \frac{\partial}{\partial x_i}(p-p_0) = 0\,. \tag{8}$$

In addition, we define the acoustic pressure p', the acoustic density ρ' and its relation (just fulfilled for linear acoustics)

$$p' = p - p_0\,; \quad \rho' = \rho - \rho_0\,; \quad \frac{p'}{\rho'} = c_0^2 \tag{9}$$

with c_0 the speed of sound. Substituting $p - p_0$ in (Eq. (8)) by $c_0^2\rho'$ and applying $\partial/\partial x_i$ to this equation, results in

$$\frac{\partial}{\partial t}\frac{\partial}{\partial x_i}(\rho v_i) + \frac{\partial^2}{\partial x_i^2}(c_0^2\rho') = 0\,. \tag{10}$$

According to the mass conservation equation, we may rewrite (Eq. (10)) as

$$\frac{\partial^2 \rho'}{\partial t^2} + \frac{\partial^2}{\partial x_i^2}(c_0^2\rho') = \left(\frac{1}{c_0^2}\frac{\partial^2}{\partial t^2} - \frac{\partial^2}{\partial x_i^2}\right)(c_0^2\rho') = 0\,. \tag{11}$$

Since we have neglected the flow and there are no other excitations, no acoustic sound field will be generated, since the solution of (Eq. (11)) is $\rho' = \rho - \rho_0 = 0$.

Lighthill's analogy is based on the fact, that the sound generated by the flow in a real fluid is exactly equivalent to that produced in the ideal, linear acoustic field, forced by the stress distribution

$$
\begin{aligned}
L_{ij} &= \tau_{ij}^{\mathrm{I}} - \tau_{ij}^{0} \\
&= \rho v_i v_j + \left((p - p_0) - c_0^2(\rho - \rho_0)\right)\delta_{ij} - \tau_{ij}\,,
\end{aligned} \tag{12}
$$

where $[L]$ denotes the Lighthill stress tensor.

We can now rewrite (Eq. (6)) as the momentum equation for the ideal, linear acoustic medium subjected to the externally applied stress according to (Eq. (12))

$$
\frac{\partial \rho v_i}{\partial t} + \frac{\partial \tau_{ij}^{0}}{\partial x_j} = -\frac{\partial(\tau_{ij}^{\mathrm{I}} - \tau_{ij}^{0})}{\partial x_j} \tag{13}
$$

Therewith, we obtain

$$
\frac{\partial \rho v_i}{\partial t} + \frac{\partial}{\partial x_i}\left(c_0^2(\rho - \rho_0)\right) = -\frac{\partial L_{ij}}{\partial x_j}\,. \tag{14}
$$

Eliminating the momentum density ρv_i similarly as for the linear case, we obtain

$$
\left(\frac{1}{c_0^2}\frac{\partial^2}{\partial t^2} - \frac{\partial^2}{\partial x_i^2}\right)\left(c_0^2(\rho - \rho_0)\right) = \frac{\partial^2 L_{ij}}{\partial x_i \partial x_j}\,. \tag{15}
$$

The integral formulation of (Eq. (15)) is given by

$$
c_0^2(\rho - \rho_0)(\mathbf{x}, t) = \frac{1}{4\pi}\frac{\partial^2}{\partial x_i \partial x_j}\int\limits_{\Omega_1} \frac{\partial L_{ij}(\boldsymbol{\xi}\,, t - r/c_0)}{r}\,\mathrm{d}\xi\,\mathrm{d}\eta\,\mathrm{d}\zeta \tag{16}
$$

with the source coordinates $\boldsymbol{\xi} = (\xi, \eta, \zeta)^T$, the observation coordinates $\mathbf{x} = (x_1, x_2, x_3)^T$ and the distance vector $\mathbf{r} = (x_1 - \xi, x_2 - \eta, x_3 - \zeta)^T$.

The integral formulation given in (Eq. (16)) is no longer correct, if any solid and/or elastic bodies (in rest or moving) are present within the simulation domain. For such situations, the acoustic field has to fulfill the correct boundary condition and we no longer can use Green's function for free radiation. Now, the idea is, to derive an extended form of Lighthill's equation, which is defined in the whole simulation domain, and thus Green's function for free radiation can be applied to obtain an integral formulation [8].

We will assume a domain with a solid body, which is located within the considered domain Ω_1. The volume of the body is denoted by Ω_{B} and its surface by Γ_{S} (see Fig. 5.1). We describe the surface of the body with a function $f(\mathbf{x}, t)$

$$
f(\mathbf{x}, t) = \begin{cases} f < 0 & \text{for } \mathbf{x} \in \Omega_{\mathrm{B}} \\ f > 0 & \text{for } \mathbf{x} \in \Omega_1 \setminus \Omega_{\mathrm{B}} \\ f = 0 & \text{for } \mathbf{x} \text{ on } \Gamma_{\mathrm{S}} \end{cases} \tag{17}
$$

e.g., a breathing sphere: $f(\mathbf{x}, t) = |\mathbf{x} - \mathbf{x}_o|^2 - R_0^2(t)$. Additionally, we introduce the Heaviside function $H(f(\mathbf{x}, t))$

$$
H(f(\mathbf{x}, t)) = \begin{cases} 1 & \text{for } \mathbf{x} \in \Omega_1 \setminus \Omega_{\mathrm{B}} \\ 0 & \text{for } \mathbf{x} \in \Omega_{\mathrm{B}} \quad . \end{cases} \tag{18}
$$

First of all, we multiply the continuity equation (see (Eq. (1))) with $H(f)$

$$
\left(\frac{\partial \rho'}{\partial t} + \frac{\partial}{\partial x_i}(\rho v_i)\right) H(f) = 0 \tag{19}
$$

and rewrite it in the following form

$$
\frac{\partial}{\partial t}\left(\rho' H(f)\right) - \rho'\frac{\partial H(f)}{\partial t} + \frac{\partial}{\partial t}\left((\rho v_i) H(f)\right) - (\rho v_i)\frac{\partial H(f)}{\partial x_i} = 0\,. \tag{20}
$$

The Heaviside function $H(f)$ fulfills two important properties [9]

$$
\frac{\partial H(f)}{\partial x_i} = \frac{\partial f}{\partial x_i}\delta(f) \tag{21}
$$

$$
\frac{\partial H(f)}{\partial t} = \delta(f)\,\frac{\partial f}{\partial t} = -v_i^{\mathrm{B}}\frac{\partial f}{\partial x_i}\delta(f) \tag{22}
$$

with v_i^{B} the velocity of the solid body in Ω_1. Using (Eq. (21)) and (Eq. (22)) simplifies (Eq. (20)) to

$$\frac{\partial(\rho' H(f))}{\partial t} + \frac{\partial(\rho v_i H(f))}{\partial x_i} = \left(\rho(v_i - v_i^{\mathrm{B}}) + \rho_0 v_i^{\mathrm{B}}\right) \frac{\partial f}{\partial x_i} \delta(f)\,. \tag{23}$$

This equation is an extension of the continuity equation, which is fulfilled all over Ω. As can be seen, the right hand side of (Eq. (23)) is just on Γ_{S} different from zero and the left side is zero within Ω_{B}. Applying similar operations to the momentum conservation equation, we obtain

$$\begin{aligned}\frac{\partial}{\partial t}\left(\rho v_i H(f)\right) &+ \frac{\partial}{\partial x_j}\left(\rho v_i v_j + p'\delta_{ij} - \tau_{ij}\right) H(f) \\ &= \left(\rho v_i (v_j - v_j^{\mathrm{B}}) + p'\delta_{ij} - \tau_{ij}\right) \frac{\partial f}{\partial x_j} \delta(f)\,.\end{aligned} \tag{24}$$

Using the definition of the Lighthill tensor L_{ij} (see (Eq. (12))), we may write (Eq. (24)) as

$$\begin{aligned}\frac{\partial}{\partial t}\left(\rho v_i H(f)\right) &+ \frac{\partial}{\partial x_i}(c_0^2 \rho' H(f)) \\ &= -\frac{\partial L_{ij} H(f)}{\partial x_j} + \left(\rho v_i (v_j - v_j^{\mathrm{B}}) + p'\delta_{ij} - \tau_{ij}\right) \frac{\partial f}{\partial x_j} \delta(f)\,.\end{aligned} \tag{25}$$

Furthermore, we rewrite (Eq. (23)) in the following form

$$\frac{\partial(\rho v_i H(f))}{\partial t} = \left(\rho(v_i - v_i^{\mathrm{B}}) + \rho_0 v_i^{\mathrm{B}}\right) \frac{\partial f}{\partial x_i} \delta(f) - \frac{\partial(\rho' H(f))}{\partial x_i}\,. \tag{26}$$

Applying $\partial/\partial x_i$ to (Eq. (25)) and using the relation according to (Eq. (26)), we arrive at the extended form of Lighthill's equation

$$\begin{aligned}&\left(\frac{1}{c_0^2}\frac{\partial^2}{\partial t^2} - \frac{\partial^2}{\partial x_i^2}\right)\left(c_0^2 \rho'\, H(f)\right) \\ =\ & \frac{\partial^2}{\partial x_i \partial x_j}\left(L_{ij} H(f)\right) - \frac{\partial}{\partial x_i}\left(\left(\rho v_i (v_j - v_j^{\mathrm{B}}) + p'\,\delta_{ij} - \tau_{ij}\right) \frac{\partial f}{\partial x_j}\delta(f)\right) \\ & + \frac{\partial}{\partial t}\left(\left(\rho(v_i - v_i^{\mathrm{B}}) + \rho_0 v_i^{\mathrm{B}}\right) \frac{\partial f}{\partial x_i}\delta(f)\right)\,.\end{aligned} \tag{27}$$

Since the extended form of Lighthill's equation is valid throughout the whole domain Ω, we can use Green's function for free radiation to obtain a solution. Therefore, the integral form, which is known as the *Ffowcs Williams and Hawkings equation* [8], reads as

$$\begin{aligned}\rho'\, H(f) = \frac{\partial^2}{\partial x_i \partial x_j} \int\limits_{\Omega_1} \frac{\partial L_{ij}}{4\pi\, r}\, d\Omega &- \frac{\partial}{\partial x_i} \oint\limits_{\Gamma_{\mathrm{S}}} \frac{\left(\rho(v_i - v_i^{\mathrm{B}}) + p'\,\delta_{ij} - \tau_{ij}\right)}{4\pi\, r}\, \mathrm{d}\Gamma \\ &+ \frac{\partial}{\partial t} \oint\limits_{\Gamma_{\mathrm{S}}} \frac{\left(\rho(v_i - v_i^{\mathrm{B}}) + \rho v_i^{\mathrm{B}}\right)}{4\pi\, r}\, \mathrm{d}\Gamma\,.\end{aligned} \tag{28}$$

5.3 Finite Element Formulation

We will perform a volume discretization of Lighthill's equation (see (Eq. (15))) by applying the finite element method (FEM). Therewith, any solid/elastic body will be implicitly taken into account, and there is no need to use the extended form of Lighthill's equation as given by (Eq. (27)).

Let us start at the strong formulation of (Eq. (15)), which reads as follows

$$
\begin{array}{lll}
\text{Given:} & L_{ij} & : \Omega \times (0,T) \to \mathbb{R} \\
 & c_0 & : \Omega \to \mathbb{R} \\
 & \rho'^0 & : \Omega \to \mathbb{R} \\
 & \dot{\rho}'^0 & : \Omega \to \mathbb{R} \\
\text{Find:} & \rho' & : \bar{\Omega} \times [0,T] \to \mathbb{R}
\end{array}
$$

$$
\begin{aligned}
\frac{\partial^2 \rho'}{\partial t^2} - c_0^2 \frac{\partial^2 \rho'}{\partial x_i^2} &= \frac{\partial^2 L_{ij}}{\partial x_i \partial x_j} \\
\rho'(\mathbf{r},0) &= {\rho'}^0 \,,\ \mathbf{r} \in \Omega \\
\dot{\rho}'(\mathbf{r},0) &= \dot{\rho'}^0 \,,\ \mathbf{r} \in \Omega
\end{aligned} \tag{29}
$$

At this stage we assume, that the Lighthill tensor $[L]$ is a known quantity obtained by a flow computation using a large eddy simulation (LES) or other appropriate type of CFD scheme. In a first step, we multiply (Eq. (29)) by an appropriate test function w and integrate over the whole domain Ω (corresponding in Fig. 5.1 to $\Omega_1 \cup \Omega_2$)

$$
\int\limits_{\Omega} w \left(\frac{\partial^2 \rho'}{\partial t^2} - c_0^2 \frac{\partial^2 \rho'}{\partial x_i^2} - \frac{\partial^2 L_{ij}}{\partial x_i \partial x_j} \right) \mathrm{d}\Omega = 0\,. \tag{30}
$$

Now, we apply Green's integral theorem to the second spatial derivative of ρ' as well as L_{ij}. This operation will result in the following relations

$$
\int_\Omega c_0^2\, w \frac{\partial^2 \rho'}{\partial x_i^2}\, \mathrm{d}\Omega = \int_{\Gamma_\mathrm{S} \cup \Gamma_\mathrm{I}} w \frac{\partial \rho'}{\partial n}\, \mathrm{d}\Gamma - \int_\Omega c_0^2 \frac{\partial w}{\partial x_i} \frac{\partial \rho'}{\partial x_i}\, \mathrm{d}\Omega \tag{31}
$$

$$
\int_\Omega w \frac{\partial^2 L_{ij}}{\partial x_i \partial x_j}\, \mathrm{d}\Omega = \int_{\Gamma_\mathrm{S}} w \frac{\partial L_{ij}}{\partial x_j} n_i\, \mathrm{d}\Gamma - \int_\Omega \frac{\partial w}{\partial x_i} \frac{\partial L_{ij}}{\partial x_j}\, \mathrm{d}\Omega\,. \tag{32}
$$

We want to emphasize, that the boundary integral (Eq. (32)) is just over the surface Γ_S of any solid/elastic body, whereas in (Eq. (31)) we have to integrate over Γ_S as well as over Γ_I, which limits the computational domain (see Fig. 5.1).

Now we can substitute $\partial L_{ij}/\partial x_j$ within the first term on the right hand side of (Eq. (32)) by (Eq. (14)) and obtain

$$
\int_{\Gamma_\mathrm{S}} w \frac{\partial L_{ij}}{\partial x_j} n_i\, \mathrm{d}\Gamma = \int_{\Gamma_\mathrm{S}} w \left(-\frac{\partial \rho v_i}{\partial t} - \frac{\partial}{\partial x_i}(c_0^2 \rho') \right) n_i\, \mathrm{d}\Gamma\,. \tag{33}
$$

Since on a solid surface $v_i n_i = 0$ is fulfilled, we see, that the surface integral term over Γ_S reduces to

$$
\int_{\Gamma_\mathrm{S}} w \frac{\partial L_{ij}}{\partial x_j} n_i\, \mathrm{d}\Gamma = -\int_{\Gamma_\mathrm{S}} c_0^2\, w \frac{\partial \rho'}{\partial n}\, \mathrm{d}\Gamma\,. \tag{34}
$$

Therewith, we can rewrite (Eq. (30)) as

$$
\begin{aligned}
\int_\Omega w \frac{\partial^2 \rho'}{\partial t^2} &+ \int_\Omega c_0^2 \frac{\partial w}{\partial x_i} \frac{\partial \rho'}{\partial x_i}\, \mathrm{d}\Omega - \int_{\Gamma_\mathrm{S} \cup \Gamma_\mathrm{I}} c_0^2\, w \frac{\partial \rho'}{\partial n}\, \mathrm{d}\Gamma \\
&= -\int_\Omega \frac{\partial w}{\partial x_i} \frac{\partial L_{ij}}{\partial x_j}\, \mathrm{d}\Omega - \int_{\Gamma_\mathrm{S}} c_0^2\, w \frac{\partial \rho'}{\partial n}\, \mathrm{d}\Gamma\,.
\end{aligned} \tag{35}
$$

Combining the surface integrals results in a single boundary integral just performed over the outer boundary Γ_I, on which we apply absorbing boundary conditions of first order according to [5]. Therewith, we utilize the relation

$$
c_0 \frac{\partial \rho'}{\partial n} = \frac{\partial \rho'}{\partial t} \tag{36}
$$

and arrive at the weak form of(Eq. (29)): Find $\rho' \in H^1$ such that

$$\int_\Omega w \frac{\partial^2 \rho'}{\partial t^2} + \int_\Omega c_0^2 \frac{\partial w}{\partial x_i}\frac{\partial \rho'}{\partial x_i}\, \mathrm{d}\Omega \, - \int_{\Gamma_\mathrm{I}} c_0^2\, w \frac{\partial \rho'}{\partial n}\, \mathrm{d}\Gamma = - \int_\Omega \frac{\partial w}{\partial x_i}\frac{\partial L_{ij}}{\partial x_j}\, \mathrm{d}\Omega \tag{37}$$

for any $w \in H^1$. With H^1 we denote the Sobolev space defined as (see e.g. [2])

$$H^1 = \{u \in L^2 | \partial u / \partial x_i \in L^2\} \tag{38}$$

and L^2 the space of square integrable functions.

Using standard nodal finite elements, we approximate the continuous acoustic density ρ' as well as the test function w by

$$\rho' \quad \approx \quad {\rho'}^h = \sum_{a=1}^{n_\mathrm{eq}} N_a \rho'_a \tag{39}$$

$$w \quad \approx \quad w^h = \sum_{a=1}^{n_\mathrm{eq}} N_a w_a \, . \tag{40}$$

Thus, (Eq. (37)) is transformed to the following semidiscrete Galerkin formulation

$$\mathbf{M}\underline{\ddot{\rho}'}_{n+1} + \mathbf{C}\underline{\dot{\rho}'}_{n+1} + \mathbf{K}\underline{\rho'}_{n+1} = \underline{f}_{n+1} \tag{41}$$

with $\underline{\ddot{\rho}'} = \partial^2 \rho_\sim / \partial t^2$, $\underline{\dot{\rho}'} = \partial \rho_\sim / \partial t$, $\underline{\rho'}$ the nodal unknowns of the acoustic density and n the time step counter. The matrices as well as right hand side vector compute as follows:

$$\mathbf{M} \quad = \quad \bigwedge_{e=1}^{n_\mathrm{e}} \mathbf{m}^e \;\; ; \;\; \mathbf{m}^e = [m_{pq}] \;\; ; \;\; m_{pq} = \int\limits_{\Omega^e} N_p N_q \, \mathrm{d}\Omega \tag{42}$$

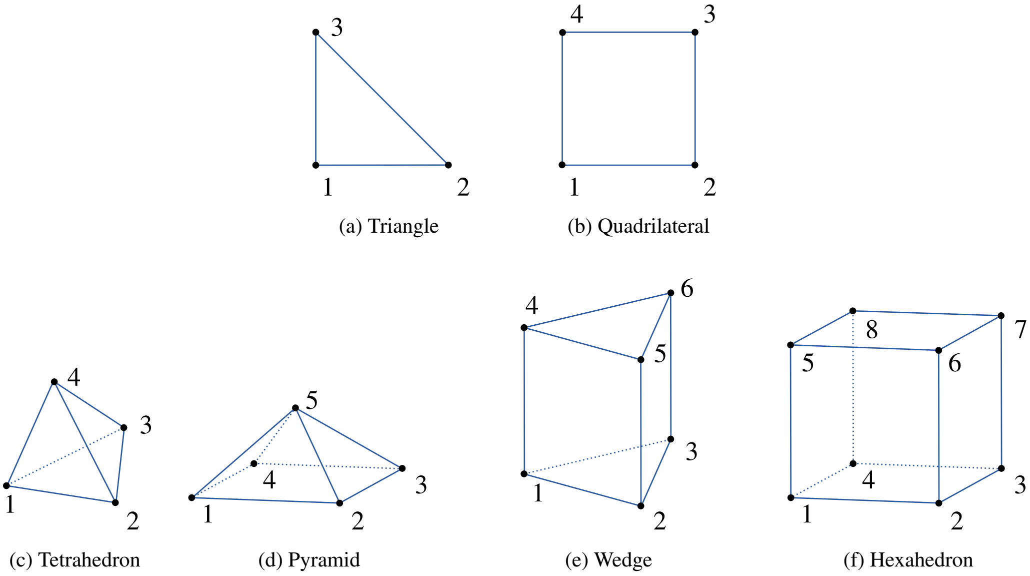
$$\mathbf{C} \quad = \quad \bigwedge_{e=1}^{n_{\Gamma_\mathrm{I}}} \mathbf{c}^e \;\; ; \;\; \mathbf{c}^e = [c_{pq}] \;\; ; \;\; c_{pq} = \int\limits_{\Gamma_\mathrm{I}^e} c_0 \, N_p N_q \, \mathrm{d}\Gamma \tag{43}$$

$$\mathbf{K} \quad = \quad \bigwedge_{e=1}^{n_\mathrm{e}} \mathbf{k}^e \;\; ; \;\; \mathbf{k}^e = [k_{pq}] \;\; ; \;\; k_{pq} = \int\limits_{\Omega^e} c^2 \, \frac{\partial N_p}{\partial x_i}\frac{\partial N_q}{\partial x_i}\, \mathrm{d}\Omega \tag{44}$$

$$\underline{f}_{n+1} \quad = \quad \bigwedge_{e=1}^{n_\mathrm{e}} \underline{f}^e \;\; ; \;\; \underline{f}^e = [f_p] \;\; ; \;\; f_p = \int\limits_{\Omega^e} \frac{\partial N_p}{\partial x_i}\frac{\partial L_{ij}^{n+1}}{\partial x_j}\, \mathrm{d}\Omega \, . \tag{45}$$

In the above equations n_e is the number of finite elements, n_{Γ_I} the number of finite surface elements along the outer boundary Γ_I and $\bigwedge$ the finite element assembly operator.

The following element types with their corresponding nodal basis functions are supported for the calculation of acoustic sources.



(a) Triangle (b) Quadrilateral

(c) Tetrahedron (d) Pyramid (e) Wedge (f) Hexahedron

Figure 2: Supported Element Types

Element type	Reference points				Nodal basis functions
	Node i	ξ	η	ζ	
Triangle	1	0	0	0	$N_1 = 1 - \xi - \eta$
	2	1	0	0	$N_2 = \xi$
	3	0	1	0	$N_3 = \eta$
Quadrilateral	1	-1	-1	0	$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta)$
	2	1	-1	0	
	3	1	1	0	
	4	-1	1	0	
Tetrahedron	1	0	0	0	$N_1 = 1 - \xi - \eta - \zeta$
	2	1	0	0	$N_2 = \xi$
	3	0	1	0	$N_3 = \eta$
	4	0	0	1	$N_4 = \zeta$
Pyramid	1	1	0	0	
	2	-1	1	0	
	3	-1	-1	0	
	4	1	-1	0	
	5	0	0	1	
Wedge	1	0	0	-1	$N_1 = (1 - \xi - \eta)(1 + \zeta_i \zeta)$
	2	1	0	-1	$N_2 = \xi(1 + \zeta_i \zeta)$
	3	0	1	-1	$N_3 = \eta(1 + \zeta_i \zeta)$
	4	0	0	1	$N_4 = (1 - \xi - \eta)(1 + \zeta_i \zeta)$
	5	1	0	1	$N_5 = \xi(1 + \zeta_i \zeta)$
	6	0	1	1	$N_6 = \eta(1 + \zeta_i \zeta)$
Hexahedron	1	-1	-1	-1	$N_i = \frac{1}{8}(1 + \xi_i \xi)(1 + \eta_i \eta)(1 + \zeta_i \zeta)$
	2	1	-1	-1	
	3	1	1	-1	
	4	-1	1	-1	
	5	-1	-1	1	
	6	1	-1	1	
	7	1	1	1	
	8	-1	1	1	

Table 2: Element types with their corresponding node coordinates and nodal basis functions

The time discretization is performed by applying a standard Newmark algorithm, which results in the following time-stepping scheme (effective mass matrix formulation) [10]:

- Perform predictor step:

$$\underline{\tilde{\rho}}' = \underline{\rho}'_n + \Delta t\, \underline{\dot{\rho}}'_n + \frac{\Delta t^2}{2}(1-2\beta_{\mathrm{H}})\, \underline{\ddot{\rho}}'_n \tag{46}$$

$$\underline{\tilde{\dot{\rho}}}' = \underline{\dot{\rho}}'^{n} + (1-\gamma_{\mathrm{H}})\Delta t\, \underline{\ddot{\rho}}'_n\,. \tag{47}$$

- Solve algebraic system of equations:

$$\mathbf{M}^* \underline{\ddot{\rho}}'_{n+1} = \underline{f}_{n+1} - \mathbf{K}\underline{\tilde{\rho}}' - \mathbf{C}\underline{\tilde{\dot{\rho}}}' \tag{48}$$

$$\mathbf{M}^* = \mathbf{M} + \gamma_{\mathrm{H}}\Delta t\, \mathbf{C} + \beta \Delta t^2\, \mathbf{K}\,. \tag{49}$$

- Perform corrector step:

$$\underline{\rho}'_{n+1} = \underline{\tilde{\rho}}' + \beta_{\mathrm{H}}\Delta t^2\, \underline{\ddot{\rho}}'_{n+1} \tag{50}$$

$$\underline{\dot{\rho}}'_{n+1} = \underline{\tilde{\dot{\rho}}}' + \gamma_{\mathrm{H}}\Delta t\, \underline{\ddot{\rho}}'_{n+1}\,. \tag{51}$$

In (Eq. (46)) - (Eq. (51)) Δt denotes the time step value and β_{H}, γ_{H} integration parameters, which are set to 0.25 and 0.5 to obtain a fully implicit time integration scheme.

5.4 Calculation Scheme

We propose a coupling between flow and acoustic computations, which allows to choose an optimal discretization for each physical field. The flow may be computed by the codes *ANSYS CFX*, *OpenFOAM*, *FASTEST-3D* [4]. *FASTEST-3D* uses a fully conservative second-order finite volume space discretization and a pressure correction method of the SIMPLE type for the iterative coupling of velocity and pressure. *ANSYS CFX* uses the Scale Adaptive Simulation (SAS) scheme. *OpenFOAM* is an open source CFD solver.

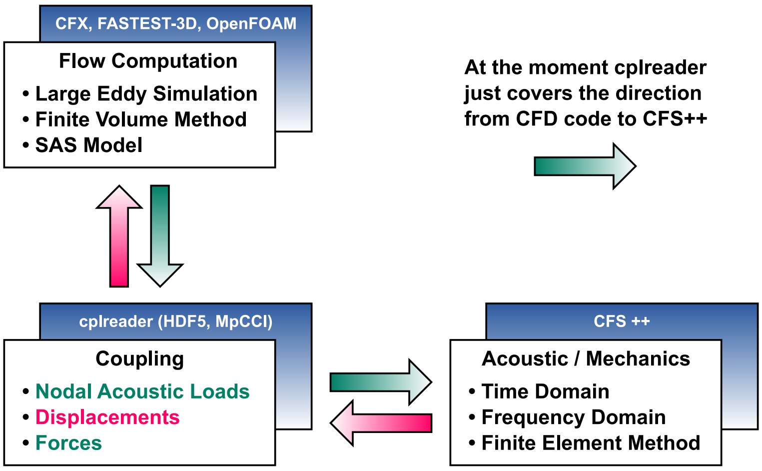


Figure 3: Coupling Scheme

For time discretization an implicit second-order scheme is employed, while a non-linear multigrid scheme, in which the pressure correction method acts as a smoother on the different grid levels, is used for convergence acceleration. The turbulent flow is computed by an appropriate CFD scheme like the large eddy simulation (LES) applying the Smagorinsky model or the SAS model of *ANSYS CFX*. The concept of block structured grids is employed in *FASTEST-3D* to handle complex geometries and for the ease of parallelization. The parallel implementation is based on grid partitioning with automatic load balancing and follows the message-passing concept, ensuring a high degree of portability.

The computation of the generated acoustic noise is performed by the finite element method as discussed in Sec. 5.3 and has been implemented in CFS++ (Coupled Field Simulation) [11]. In order to minimize the amount of data transfer and to achieve a higher accuracy, we already compute the right hand side $\underline{f}^{n+1}$ (see Eq. (45)) on the much finer flow grid and obtain so called acoustic nodal loads. This is performed as a post-processing step of the CFD computation by cplreader. Within each time step, these scalar values are interpolated to the acoustic grid by using a customized conservative interpolation algorithm in *CFS++* or MpCCI - the Mesh-based parallel Code Coupling Interface [1] (see Fig. 3). In doing so the weights for the distribution of the CFD quantity to the nodes in the acoustic cell are obtained by evaluating the basis functions defined in Tab. 2 at the position of the CFD node.

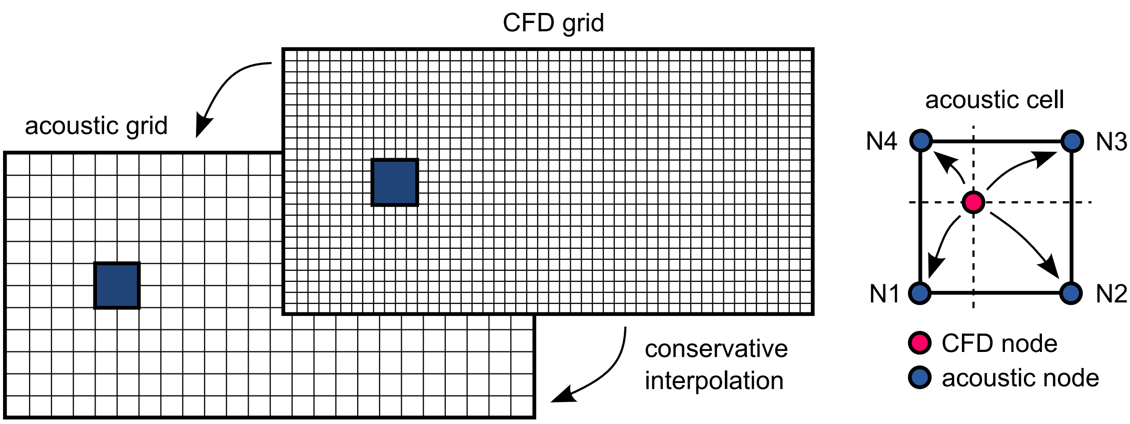


Figure 4: Conservative interpolation of source terms from CFD to acoustic grid.

The data transfer has to be performed within the whole computational domain of the flow (corresponds to Ω_1 in Fig. 5.1). In order to obtain a correct interpolation, the acoustic nodal loads have then to be interpolated by a *conservative technique*, since they correspond to an integral quantity.

Therewith, the developed scheme allows a direct coupling in time domain and provides the acoustic sound field not only in the far field but also in the region of the flow. Furthermore, we can directly investigate the acoustic source terms in the flow region and the scheme is well suited for complex geometries and fluid structure interaction. By applying a discrete Fourier transform to the acoustic nodal loads, it is also possible to perform analyses in the frequency domain.

In addition to the full 3D coupling, we have implemented a 2D coupling from the flow to the acoustic computation. Therewith, we just compute the acoustic nodal loads on a predefined 2D slice and perform the acoustic field computation only within a cross section. It is also possible to slice a 3D CFD dataset by interpolating to a 2D acoustic grid. In both cases the acoustic nodal loads have been previously computed and written to a *HDF5* file by cplreader. This file has then been read and interpolated by *CFS++* (cf. Fig. 4). This technique allows us to write the computed acoustic nodal loads to a file and perform acoustic simulations independent of the flow computation.

6 CFD Data Generators

The first purpose of cplreader when started by Max Escobar was to just read velocity fields generated by FASTEST-3D and to provide this data to CFS++ via an MpCCI interface. Since then cplreader has evolved from just a simple reader to an intermediate program which can also calculate acoustic source terms.

These source terms can also be generated by cplreader from analytic expressions. This has however not been implemented yet. Possible extensions for cplreader include:

- Max Escobars vortex core model from AcouFlowNoise.cc
- Source term reader for AcouCombustion

7 CFD Data Readers

7.1 FASTEST-3D

7.1.1 Required File Layout

```
./name
|-- name_01.coord
|-- name_01.topo
|-- name_01_0001.dat
|-- name_01_0002.dat
|-- name_02.coord
|-- name_02.topo
|-- name_02_0001.dat
`-- name_02_0002.dat
```

7.1.2 Special Options

7.1.3 ASCII File Format

`.coord`-File Format

```
 1   1   2   6
  0.  0.  0.0
  0.  0.03125  0.0
  0.  0.09375  0.0
  0.03125  0.  0.0
  0.03125  0.03125  0.0
  0.03125  0.09375  0.0
```

`.topo`-File Format

```
  1 1 2 2 4
 4 5 2 1
 5 6 3 2
```

`.dat`-File Format

```
      1
 -5.30794066E-06
 -1.39465176E-05
 -2.06977642E-05
 -1.94136509E-05
 -3.94903112E-05
 -7.02913899E-05
```

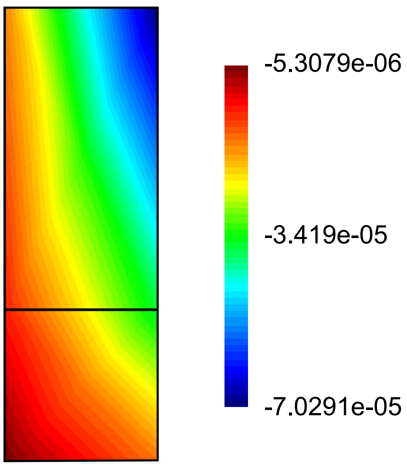


Figure 5: Visualization of dataset defined by above files.

7.1.4 Examples

7.2 ANSYS CFX

7.2.1 Required File Layout

```
./name/
|-- name
|   |-- 35445.trn
|   `-- 35446.trn
|-- name.def
`-- name.res
```

All entities 'name' may also be symbolic links.

7.2.2 Special Options

–type The type of input files have to be set to 'CFX'.

–deffile You can specify different '.def' file than the one given by 'name'.

–trntol Percentage the file size of transient files may differ from the average file size of all transient files. If a transient file is smaller than average size * (1 - trntol) the corresponding time step will be overwritten by the previous one.

7.2.3 Logging

Opening the root *HDF5*file ('name'.h5) one can find in /UserData/CFX_COMMANDS information about the CFX run which produced the .res file. This includes timestep size, material properties, boundary conditions, turbulence model, etc. The string dataset /UserData/TRN_TO_STEP_MAP contains information about which .trn file was mapped to which timestep. This is useful if corrupt .trn files have been encountered which got replaced by the last previous non-corrupt .trn file.

7.2.4 Examples

Let us assume, that we have got a directory layout like the following:

```
.
|-- fluid_old.def
|-- fluid_001
|   |-- 0.trn          1000 KB
|   |-- 1.trn          1000 KB
|   |-- 2.trn           950 KB
|   |-- 3.trn          1000 KB
|   |-- 4.trn          1000 KB
|   |-- 5.trn          1000 KB
|   |-- 6.trn           900 KB
|   |-- 7.trn          1000 KB
|   |-- 8.trn          1000 KB
|   |-- 9.trn          1000 KB
|   |-- 10.trn         1000 KB
|     ...
`-- fluid_001.res
```

We first have to rename or relink all entities 'fluid_001' to 'fluid' so that they have consistent names. We do not have a 'fluid.def' where the settings of the CFX simulation are saved. But let's also assume you have a 'fluid_old.def' which has the same grid information in it. Such a '.def' file could have been generated by a restarted CFX run. You may also use this '.def' file for reading the grid information. Now you could start reading in the data by issuing:

```
cplreader --type CFX --name fluid --numsteps 11 --timestep 1e-4 \
          --deffile fluid_old.def --trntol 0.05 --outputfields 'all'
```

This would read all transient files except 6.trn which would be replaced by 5.trn. You can even replace the .def file by the .res file since the grid is also saved in .res. The transient files get searched by cplreader in this case.

Please note that no surface elements will be read by the CFX reader.

7.3 CFX_EXPORT

7.3.1 Required File Layout

```
./name/
|--results_1.csv
`--results_2.csv
```

7.3.2 Required File Format

Data is stored in *groups*, that must begin with a line like `[Name]`. Group names are case sensitive. Groups not mentioned here are ignored. Lines that begin with `#` are ignored.

`Data` group This group contains the coordinates of nodes as well as the flow data per node. The first line contains the column headers separated by comma. Each column header consists of a name of a field followed by its unit enclosed in brackets, e.g. "`Pressure [ Pa ]`". Table 3 shows the recognized column headers. All following lines up to the next group (or end of file) provide the data using one line per node and values separated by comma.

Table 3: Data fields supported by CFX_EXPORT reader

Field name	Unit	Description
`Node Number`		non-negative unique integer number identifying the node
`X`	`m`	x coordinate of the node (required)
`Y`	`m`	y coordinate of the node (required)
`Z`	`m`	z coordinate of the node
`Density`	`kg m^-3`	
`Pressure`	`Pa`	
`Velocity u`	`m s^-1`	x component of velocity in computational frame
`Velocity v`	`m s^-1`	y component of velocity in computational frame
`Velocity w`	`m s^-1`	z component of velocity in computational frame

`Faces` group This group contains the connectivity of nodes. There must be no column header line. Each line provides the node numbers (as defined in the `Data` group) of one finite element, separated by comma. Elements with 3, 4, 5, or 6 nodes are allowed. Elements with 5 or 6 nodes will be split into triangles.

Although every data file may contain different node coordinates and different connectivity, the mesh is only read from the first data file.

7.3.3 Examples

```
[Data]
Node Number, X [ m ], Y [ m ], Velocity u [ m s^-1 ], Velocity v [ m s^-1 ]
0, 0.000e+00, 0.000e+00, 4.200e+01, 1.000e+01
1, 0.000e+00, 3.125e-02, 4.200e+01, 1.200e+01
2, 0.000e+00, 9.375e-02, 4.200e+01, 1.100e+01
3, 3.125e-02, 0.000e+00, 4.200e+01, 0.000e+00
4, 3.125e-02, 3.125e-02, 4.200e+01, 2.000e+00
5, 3.125e-02, 9.375e-02, 4.200e+01, 1.000e+00

[Faces]
3, 4, 1, 0
4, 5, 2, 1
```

7.4 OpenFOAM

7.4.1 Required File Layout

```
./name/
|-- 0
|    |-- U
|    |-- motionDiff
|    |-- motionU
|    `-- p
|-- 0.01
|    |-- U.gz
|    |-- motionU.gz
|    |-- p.gz
|    `-- polyMesh
|-- 0.02
|    |-- U.gz
|    |-- motionU.gz
|    |-- p.gz
|    `-- polyMesh
|-- constant
|-- log
`-- system
```

7.4.2 Special Options

7.4.3 Logging

Opening the root *HDF5*file ('name'.h5) one can find in /UserData all files from the directory structure of OpenFOAM which are necessary to restart the OpenFOAM simulation. This includes the files in the following directories:

```
./name/
|-- 0
|-- constant
|-- log
`-- system
```

Informations like timestep size, mesh, turbulence model, etc. should be available from these files.

7.4.4 Examples

Stefan Zörner has got plenty of examples. Or you may have a look at /home/lse24/lse/strieben/cplreader_performance_suite.

8 Other Readers and Mesh Generators

8.1 ANSYS Mesh Converter

The ANSYS mesh converter can read either ANSYS mesh output files generated by the older mkmesh extension and convert them to *HDF5*or the the files generated by the NACS GiD/ANSYS extension. At LSE a version of cplreader is used to provide the ANSYS mkhd5 (/home/data/programs/mkhdf5) APDL extension.

8.2 Mesh Generators

The syntax for calling cplreader is:

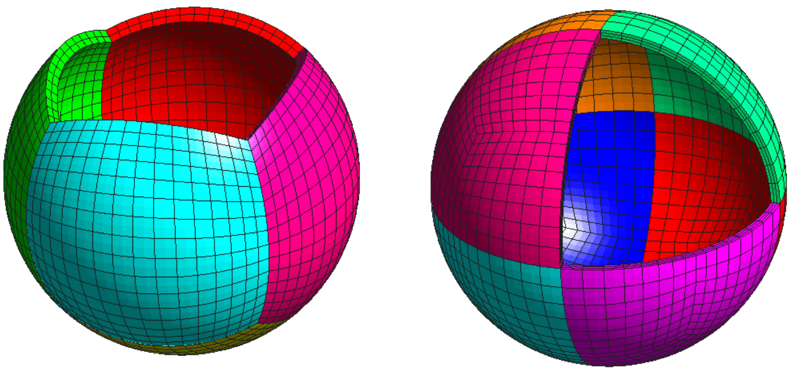
```
cplreader --name test --type GENGRIDS --xmlfile test.xml
```

8.2.1 Spherical Shells

I (Simon Triebenbacher) implemented this grid generator to try out spherical shell grids composed of all available element types (including HEXA27) for nonmatching grids. It can be only used through a settings XML file at the moment. Here is an example for an XML file for generating spherical shells.

```
<cplreaderOptions>
  <type name="GENGRIDS" subType="SphericalShellGenerator">
    <elementType name="HEXA27"/>
    <numRefinementLevels value="3"/>
    <outerRadius value="0.95"/>
    <innerRadius value="0.9"/>
    <numRadialElements value="3"/>
    <octantRegions value="off"/>
  </type>
</cplreaderOptions>
```

Figure 6 shows the influence of the 'octantRegions' parameter on the distribution of the regions inside the generated grid.



(a) Default setting (octantRegions off) (b) Using octant regions (octantRegions on)

Figure 6: Influence of the 'octantRegions' parameter.

8.2.2 Rectilinear Grids

Cplreader has been extended with a mesh generator for generating simple rectilinear meshes consisting of hexahedra elements. The geometry for the setup may only consist of a grid of boxes. The geometry is described by a number of intervals in x-, y-, and z-directions. In each interval a number of subdivisions may be specified (similar to ESIZE,,NDIV in ANSYS). Cplreader treats each of the boxes as volumes (in ANSYS terminology). The naming convention is 'volume_x_y_z' as can be seen for a simple example in Fig. 8.2.2.

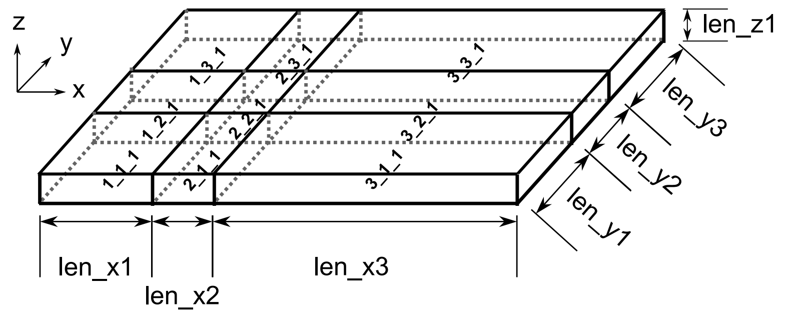


Figure 7: Naming conventions for volumes.

For each volume six surfaces are being automatically generated using the naming convention given in Fig. 8.2.2.

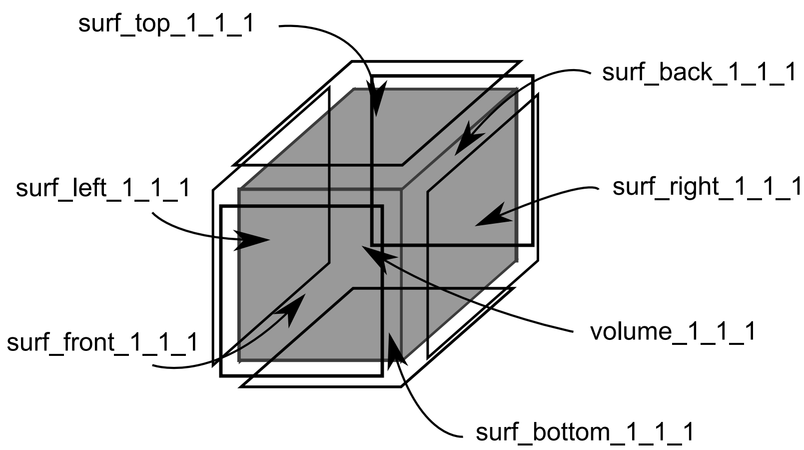


Figure 8: Naming conventions for surfaces.

For the bottom layer surface elements also line segments are create following the naming convention 'lines_[left|right|front|back]'. Due to this one can also create 2D meshes with line elements on the boundaries. Here is a sample XML file.

```
<cplreaderOptions>
  <type name="GENGRIDS">
    <elementType name="HEXA8"/>
    <samplingIntervalls>
      <!-- Define intervals by using either absolute or incremental coordinates -->
      <coordinates axis="x">
        <coord type="absolute">  0.0e-0 </coord>
        <coord type="increment"> 2.0e-1 </coord>
        <coord type="increment"> 5.0e-2 </coord>
        <coord type="increment"> 2.0e-1 </coord>
      </coordinates>
      <coordinates axis="y">
        <coord type="absolute">  0.0e-0 </coord>
        <coord type="increment"> 1.0e-1 </coord>
        <coord type="increment"> 2.0e-2 </coord>
        <coord type="increment"> 1.0e-1 </coord>
      </coordinates>
      <coordinates axis="z">
        <coord type="absolute">  0.0e-3 </coord>
```

```
        <coord type="increment"> 5e-3 </coord>
      </coordinates>
    </samplingIntervalls>
    <!-- Assign number of elements to each interval -->
    <intervallElements>
      <elements axis="x">
        <elems>2</elems>
        <elems>5</elems>
        <elems>10</elems>
      </elements>
      <elements axis="y">
        <elems>2</elems>
        <elems>7</elems>
        <elems>12</elems>
      </elements>
      <elements axis="z">
        <elems>2</elems>
      </elements>
    </intervallElements>
    <regions>
      <!-- Define regions by selecting volumes, surfaces and lines -->
      <region name="region1">
        <prelimRegion> volume_1_1_1 </prelimRegion>
        <prelimRegion> volume_2_1_1 </prelimRegion>
        <prelimRegion> volume_3_1_1 </prelimRegion>

        <prelimRegion> volume_1_2_1 </prelimRegion>
        <prelimRegion> volume_3_2_1 </prelimRegion>

        <prelimRegion> volume_1_3_1 </prelimRegion>
        <prelimRegion> volume_2_3_1 </prelimRegion>
        <prelimRegion> volume_3_3_1 </prelimRegion>
      </region>

      <region name="region2">
        <prelimRegion> volume_2_2_1 </prelimRegion>
      </region>

      <region name="surf_region2_bottom">
        <prelimRegion> surf_bottom_2_2_1 </prelimRegion>
      </region>

      <region name="left_lines">
        <prelimRegion> lines_left_1_1_1 </prelimRegion>
        <prelimRegion> lines_left_1_2_1 </prelimRegion>
        <prelimRegion> lines_left_1_3_1 </prelimRegion>
      </region>
    </regions>
  </type>
</cplreaderOptions>
```

Figure 8.2.2 shows the grid generated from the above XML file.

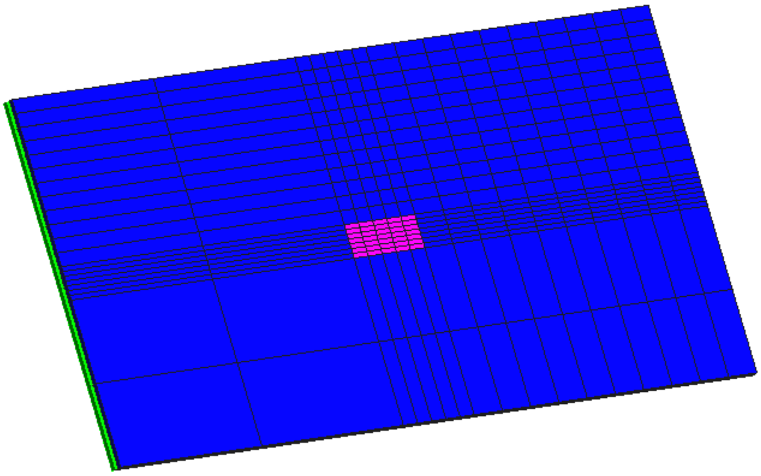


Figure 9: Rectilinear grid generated from sample XML file.

8.2.3 Reference Elements

This generator just outputs a single reference element of specified type using the coordinates and node numbering as used internally within CFS++. The following gives an example of generating a wedge element.

```
<cplreaderOptions>
  <type name="GENGRIDS" subType="ReferenceElementGenerator">
    <elementType name="WEDGE15"/>
  </type>
</cplreaderOptions>
```

Figure 8.2.3 shows the available element types file.

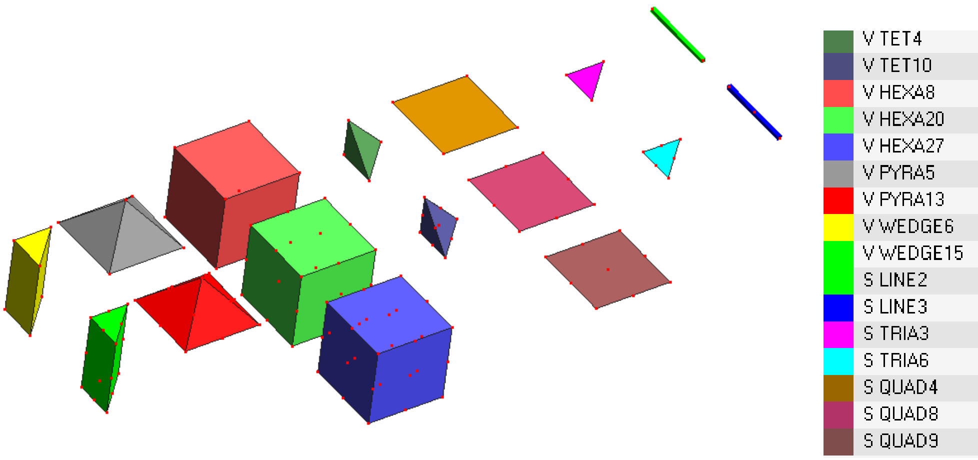


Figure 10: Array of reference elements from CFS++.

9 Rebuilding the Documentation

Switch on `CPLREADER_REBUILD_DOCU` in CMake GUI. Make sure you have a recent TikZ/PGF distribution installed. Edit the following files:

```
source/cplreader/Documentation/latex/cplreader_docu.tex
source/cplreader/Documentation/latex/Title.tex
source/cplreader/cmake_options.cmake
```

Then you may rebuild CFS++ and cplreader. The resulting PDF documentation file for cplreader will be copied back to the source directory automatically. Therefore you just need to commit the changes to SVN.

10 Copyright

11 See Also

Homepage of the LSE: http://www.lse.uni-erlangen.de

Homepage of the Chair for Applied Mechatronics:
http://www.uni-klu.ac.at/tewi/ict/sst/am/index.html

Homepage of the Simetris GmbH: http://www.simetris.de

Hierarchical Data Format page: http://hdf.ncsa.uiuc.edu/HDF5

OpenFOAM: http://www.opencfd.co.uk/openfoam

ANSYS CFX: http://www.ansys.com/Products/cfx.asp

Computational Aeroacoustics in Erlangen: http://www.lstm.uni-erlangen.de/aeroakustik/

Computational Fluid Dynamics at LSTM Erlangen
http://www.lstm.uni-erlangen.de/Ber/BerT1/index-en.htm

Chair of Fluid Mechanics and Fluid Machines, University Freiberg
http://www.imfd.tu-freiberg.de/fluid/index.en.html

References

[1] MpCCI - Mesh-based Parallel Code Coupling Interface. http://www.mpcci.org/, 2003.

[2] R.A. Adams. *Sobolev Spaces*. Pure and Applied Mathematics. Academic Press, 1975.

[3] T. Colonius and J.B. Freund. Application of Lighthill's eqation to mach 1.92 turbulent jet. *AIAA Journal*, 38(2):368–370, 2000.

[4] F. Durst and M. Schäfer. A parallel block-structured multigrid method for the prediction of incompressible flows. *Int. J. Num. Methods Fluids*, 22:549–565, 1996.

[5] Björn Engquist and Andrew Majda. Absorbing boundary conditions for the numerical simulation of waves. *Mathematics of Computation*, 31(139):629–651, 1977.

[6] Max Escobar. *Finite Element Simulation of Flow-Induced Noise using Lighthill's Acoustic Analogy*. PhD thesis, University Erlangen-Nuremberg, 2007.

[7] R. Ewert, M. Meinke, and W. Schröder. Comparison of source term formulations for a hybrid CFD/CAA method. In *7th AIAA/CEAS Aeroacoustics Conference, Maastricht, The Netherlands*, may 2001.

[8] J.E. Ffowcs-Williams and D.L. Hawkings. Sound radiation from turbulence and surfaces in arbitrary motion. *Phil. Trans. Roy. Soc. A*, 264:321–342, 1969.

[9] M.S. Howe. *Theory of Vortex Sound*. Cambridge University Press, 2002.

[10] M. Kaltenbacher. *Numerical Simulation of Mechatronic Sensors and Actuators*. Springer, Berlin, 2. edition, 2007.

[11] M. Kaltenbacher, A. Hauck, M. Hofer, M. Mohr, and E. Zhelezina. CFS++: Coupled Field Simulation. Technical report, LSE, 2005.

[12] M. Kaltenbacher, M.Escobar, I. Ali, and S. Becker. Finite element formulation of Lighthill's analogy. Technical report, Chair of Sensor Technology, Erlangen, 2005.

[13] M.J. Lighthill. On sound generated aerodynamically - I. General theory. *Proc. Roy. Soc. Lond.*, A 211(1107):564–587, 1952.

[14] M.J. Lighthill. On sound generated aerodynamically - II. Turbulence as a source of sound. *Proc. Roy. Soc. Lond.*, A 222(1148):1–22, 1954.

[15] A. A. Oberai, F. Roknaldin, and T.J.R. Hughes. Computational procedures for determining structural-acoustic response due to hydrodynamic sources. *Computer Methods in Applied Mechanics and Engineering*, 190:345–361(17), October 2000.

[16] C.K.W. Tam. Computational aeroacoustics. *AIAA Journal*, 33:1788–1796, 1995.

[17] C.K.W. Tam. Supersonic jet noise. *Annual review of fluid mechanics*, 27:17–43, 1995.